

w

Web Application Architecture
Project Report

TU059
MSc in Computer Science
(Advanced Software Development)

To Be Peppermint

Jayam Patel - A00083216

School of Computer Science
TU Dublin – City Campus

Oct 12, 2026

Table of Contents

Project Functionality **4**

Project Architecture **4**

Monolith Architecture (Current): 4

Client-Server Architecture (Option 1, Chosen): 4

Microservices Architecture (Option 2): 5

Technology Stack **5**

Implementation Strategy **6**

Data consumption from a REST API **6**

Advanced search and filter mechanisms **6**

Usability and user experience **7**

Code quality and design patterns **7**

Application security **8**

Conclusion **8**

Declaration

I hereby declare that the work described in this project is, except where otherwise stated, entirely my/our own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Jayam Patel

Oct 12, 2026

- ☐ ***Add description of the existing system and issues with it. Needs Attention.***
- ☐ ***Add more description as if non-technical person is reading it. Like Google Summer of Code proposal. Needs Attention.***

Project Functionality

- ☐ ***Divide requirements in functional and non-functional requirements. Needs Attention.***

Items to address:

- Describe the functionality on offer in the application.
- For the group projects detail who worked on / completed each of these features.

- The application should allow users to create/read/update/delete (CRUD operations) their stock investments
- Change in net-worth snapshot.
- Feature private stock investments
- Cashflow tracking – allow day to day transactions.
- Allow credit/debit from multiple accounts in a single transaction.
- Allow multiple categories in a single cashflow transaction.

Project Architecture

Items to address:

- Issues with the existing monolith. Detailed description of the monolith and why it is advisable to move from this architecture
- New Architecture – what was reviewed and detail why this architecture was chosen. Full detailed description of the chosen architecture.

Monolith Architecture (Current):

Architecture Overview: In existing system, a monolith architecture has been used. It's a 1-layered architecture with everything coded in one place. There are two major issues with the existing system. One, given high coupling, it's hard and expensive to add new features. New features are slow to deploy and requires rigorous testing. Secondly, due to huge size of data and it's processing, it requires scaling and it is not possible to scale horizontally. Additionally, we are tied to NextJS for frontend and backend.

Tech Stack:

- NextJS for frontend and backend
- SQLite for database

Pros:

- Easier development and faster starting point

Cons:

- Fully coupled, full deployment required for each of the changes
- Difficult to scale horizontally
- Harder to manage backend with extensive logic

Client-Server Architecture (Option 1, Chosen):

Architecture Overview: The new architecture will separate frontend, backend, and database. This separation will allow us to develop frontend and backend independently. It

will also allow us to scale backend. Oracle database will allow advanced querying making mathematical calculations fast.

Tech Stack:

- Frontend: NextJS
- Backend: Python, FastAPI, module separation
- Database: Oracle Database
- Auth: Auth0 (Third-party provided)

Pros:

- Clear separation between frontend, backend, and database
- All data (coupled) is stored together, making it easier to keep data integrity
- Freedom to choose tech-stack for frontend, backend, and database
- Clear API separation, which can be used for mobile app, MCP server, and more

Cons:

- Coupled system compared to micro-service architecture

Conclusion:

- Selected this architecture as it offers modularity while keeping data at the same place.
- Scaling can be achieved by dividing users across different database as users are not liked with each other

Microservices Architecture (Option 2):

Architecture Overview: We have a separate frontend, and microservices (cashflow service, investment service, networth service, etc).

Tech Stack:

- Frontend: NextJS
- Each service has its own preferred framework and database

Pros:

- Easier to manage each of the services separately
- Easy to scale horizontally

Cons:

- Our data is tightly coupled, between investments, cashflows, and accounts. With different service for each of the logic makes it harder to manage data integrity

Conclusion:

- Rejected this architecture as cons of data integrity outweighs horizontal scaling advantage.

Technology Stack

Items to address:

- Choose the Technology stack recommendation (e.g., Node.js, Docker, Kubernetes, Redis, etc.).

- NextJS for frontend. It allows easy development using API.
- FastAPI for backend. FastAPI allows creating API easily, while Python will support proper Oracle integration.
- Oracle Database: Oracle database offers advanced querying while will improve calculation speed.

- Docker: A docker compose file with three containers for frontend, backend, and database.

Implementation Strategy

Items to address:

- Phased approach to implementing the new components and the division of work between the development team.
- Load balancing strategy and auto-scaling plans.
- Front-end design with emphasis on responsive design (React, Angular, Vue.js, or similar).
- Phase 1: Basic frontend and backend implementation.
- Phase 2: Auth and multi-user support.
- Phase 3: Query optimizing.
- Phase 4: Docker and CI/CD support.
- Phase 5: Load-balancing and auto-scaling based on users. I.e. user gets one of the backend servers with a database.

Data consumption from a REST API

Items to address:

- How was the REST API setup.
- What feature does this apply to.
- How does this follow best practices.
- How does the API deal with errors / routes.
- What security has been implemented for the APIs
- Is there any caching features?
- API should have endpoints with /cashflow, /investment, /categories, and /accounts endpoints.
- These endpoints will allow CURD operations for cashflows, investments, categories, and accounts.
- API endpoints will throw standard errors such as unauthorized access, duplicate entry, etc.
- API endpoints will verify user with their auth token. This verification will be handled by third-party app Auth0 offering Identify-as-a-service.
- API should cache read queries. This cache should auto-brust when data is updated.

Advanced search and filter mechanisms

Items to address:

- Filter panel location / positioning on the UI
- Visibility of applied filters
- Clear and descriptive labels for search filters
- Provide adequate feedback
- For large data sets is there pagination / lazy loading
- Filters should be available in cashflows. User should be able to filter account, date, and/or category using dropdown.
- User should also be able to search description.
- Adding filter should update URL with queries to make filtered pages easily.

- Given high number of transactions, they should be paginated from most recent to oldest.

Usability and user experience

Items to address:

- Intuitive design:
  - users understanding of the architecture of the site and how easily they can work with the functionality on offer.
- Ease of learning:
  - how quickly a user can learn to work with the system
- Efficiency of use:
  - How fast an experienced user can accomplish tasks
- Memorability:
  - how a user can remember how to work with the system
- Error frequency and severity:
  - how are errors (user and system) dealt with in the application.
- Subjective satisfaction:
  - Does the user like working with the system?
- Intuitive design:
  - The application should have left-navigation bar. It should allow user to easily navigate to different pages.
  - Application should have following pages: overview, investment pages (holdings and investment transactions), cashflows (cashflow overview, snaky chart, cashflow transactions, categories, and accounts), and profile and settings page.
- Given simple UI layout, it should be easier for new user to navigate to applications. Additionally, users can see documentation for proper guide adding specific/special transactions such as stock investments and transfers.
- Efficiency of use:
  - User should be able to easily add dividends using few clicks.
  - User should be able to add multiple transactions with common details.
- Memorability:
  - All UI components will have proper title and/or placeholders to allow users to easily remember flow.
- Error frequency and severity:
  - Application should have least of zero errors.
  - Given the data is user specific and manual, errors in calculations can be easily mitigated using recalculation.
- Subjective verification:
  - User likes the new UI, easy usage, and multiple transaction support.
  - User is happy with fast load times.

Code quality and design patterns

Code quality:

- Effective class, method, and variable names
- Effective top-down decomposition of algorithms
- Code layout should be readable and consistent
- Effective source tree directory structure
- Effective file organization
- Correct exception handling
- Good unit test cases

Design patterns:

- For the patterns used in the application what benefits have they brought to the implementation. List all patterns and principles used.

Code quality:

- Effective class, method, and variable names.
- Code layout should be readable and consistent.
- Correct exception handling.
- Unit tests, and end-to-end tests.

Design patterns:

- Layered Architecture (FastAPI backend): Separates API routes, business logic, and database operations, improving maintainability and testability.
- Repository Pattern (Single Repository divided into sub-repos): Repository is divided into frontend, backend, and database, separating dependencies.
- Service Layer Pattern: Keeps business logic separate from HTTP requests.
- Adapter Pattern: Auth0, external stock-price APIs, and Oracle integration - Provides a consistent internal interface for external services and simplifies replacing providers.

Application security

Suggested items to address:

- Injection
- Broken Authentication and Session Management
- Sensitive Data Exposure
- Cross-Site Scripting
- Insufficient Logging & Monitoring
- Application will use Auth0 for authentication and authorization.
- Application will also support API usage using API key for MCP server and easy injection using AI tools.

Conclusion

Summary of the implementation and achievement

In summary, new system will support horizontal scaling, separation of frontend, backend, and database. This will decouple dependencies in frontend and backend. It will also support filters, pagination, caching, and faster queries.